

GALGOTIAS COLLEGE OF ENGINEERING & TECHNOLOGY

Object Oriented Programming Lab

Subject Code: BMC252

Semester: II



Session: 2025-26

Submitted To:

Mr Md Shahajad
AP, MCA Department

Submitted By:

Anshika Trivedi
Roll No: 2500970140031

Department of Computer Applications

Knowledge Park I, Greater Noida, Uttar Pradesh 201310

INDEX

S.N	Name of the Program	Date of Implementation	Date of Submission	Sign
1	Write a program in Java to print Hello.			
2	Write a program in java to understand the difference between print () and println ()			
3	Write a program in Java with two classes create a object of first class and call into another class (having main method)			
4	Write a program in java to product of two numbers			
5	Write a program in java product of two numbers (Input by the user)			
6	Write a program in java to illustrate the concept of local, instance and static variable			
7	Write a program in java to implement implicit and explicit type casting			
8	Write a program in java to implement the various operators in java			
9	Write a program in Java for constructor overloading			
10	Write a program in Java for method overloading			
11	Write a program in Java for method overriding			
12	Write a program in java to show run time polymorphism (up casting)			
13	Write a program in java for access specifiers (all four)			
14	Write a program in java to implement the single dimension array			
15	Write a program in java to copy the elements from one array to another array			
16	Write a program in java to perform the addition and multiplication in 2-D array			
17	Write a program in Java for simple inheritance			
18	Write a program in java for Final keyword			
19	Write a program in java for super keyword			
20	Write a program in Java chaining constructor			
21	Write a program in Java for abstract method and abstract class			
22	Write a program in Java for interface			
23	Write a program in Java multiple inheritance			
24	Write a program in Java for Object Cloning (shallow and deep copy)			
25	Write a program in Java for Inner Classes (all types)			
26	Write a program in java to create the package (user defined package)			
27	Write a program in Java for exception handling by using try, catch and finally			

28	Write a program in Java for throw and throws Exception			
29	Write a program in Java to throw your own Exceptions			
30	Write a program in Java to reading and writing in file using byte stream			
31	Write a program in Java to reading and writing in file using character stream.			
32	Write a program in Java to reading and writing through console class.			
33	Write a program in Java how to create thread using Thread Class.			
34	Write a program in Java how to create thread using runnable interface.			
35	Write a program in Java to implement the multithreading.			
36	Write a program in Java to achieve synchronization in threads.			
37	Write a program in Java to implement the concept of Priorities in threads.			
38	Write a program in Java to illustrate the concept of Generic Programming			
39	Write a program in Java to illustrate the concept of event handling (using various event handlers)			
40	Create a simple student registration application using various swing components (like: JFrame, JButton, JLabel, text fields, text areas, check box and radio buttons).			

Program No. 1

Objective: Write a program in Java to print Hello.

Code Implementation (Source Code):

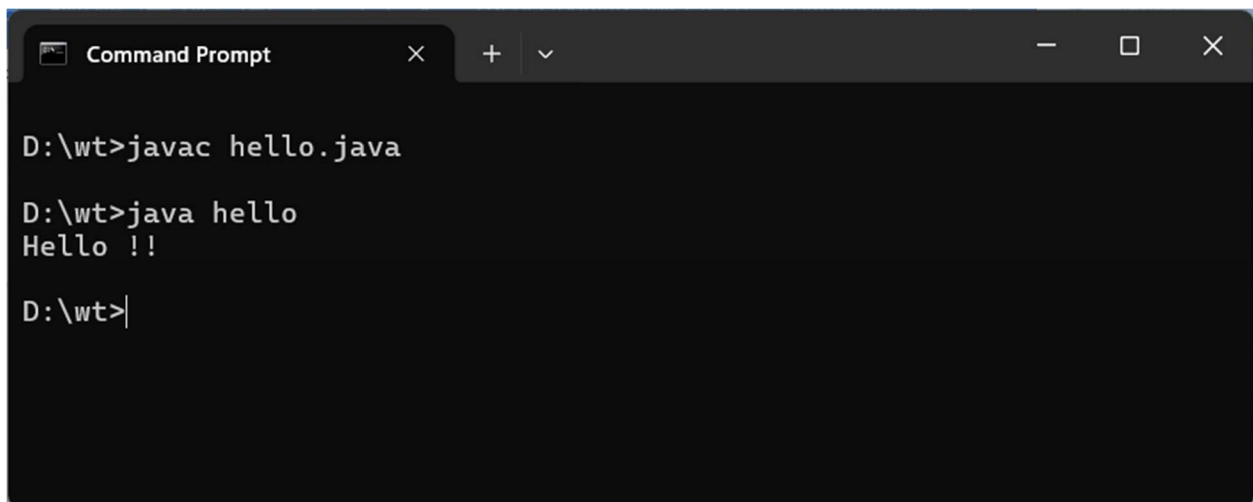
```
public class Main {  
    public static void main(String[] args) {  
        System.out.println("Hello !!");  
    }  
}
```

****Output:****

'''

Hello, World!

Output Screen:



```
Command Prompt  
D:\wt>javac hello.java  
D:\wt>java hello  
Hello !!  
D:\wt>|
```

Program No. 2

Objective: Write a program in java to understand the difference between print () and println ().

Code Implementation (Source Code):

```
public class Que2{

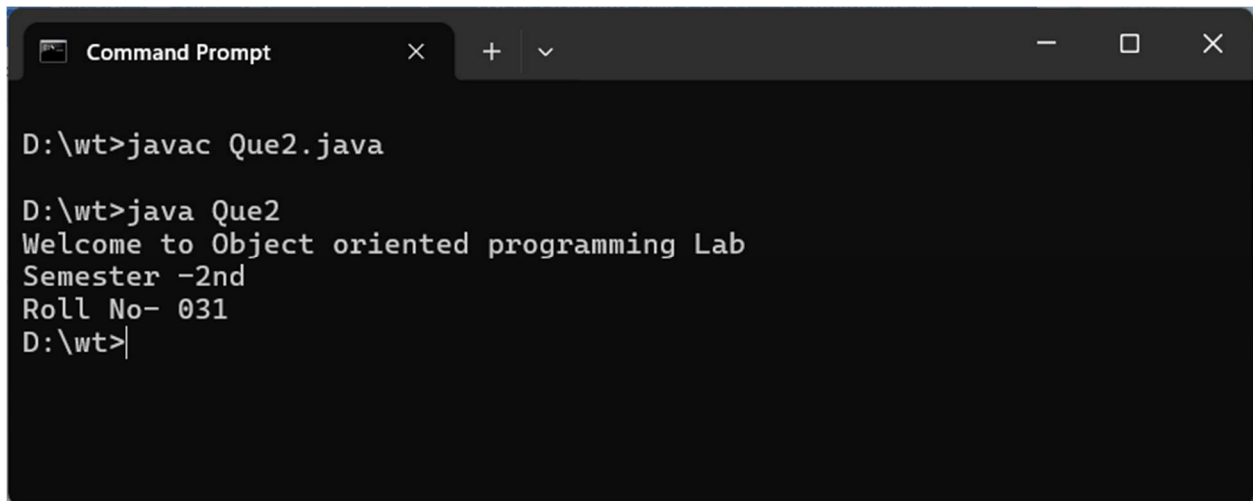
    public static void main(String [] args){

        System.out.println("Welcome to Object oriented programming Lab");
        System.out.println("Semester -2nd");
        System.out.print(" Roll No-031");

    }

}
```

Output Screen:



```
Command Prompt
D:\wt>javac Que2.java
D:\wt>java Que2
Welcome to Object oriented programming Lab
Semester -2nd
Roll No- 031
D:\wt>|
```

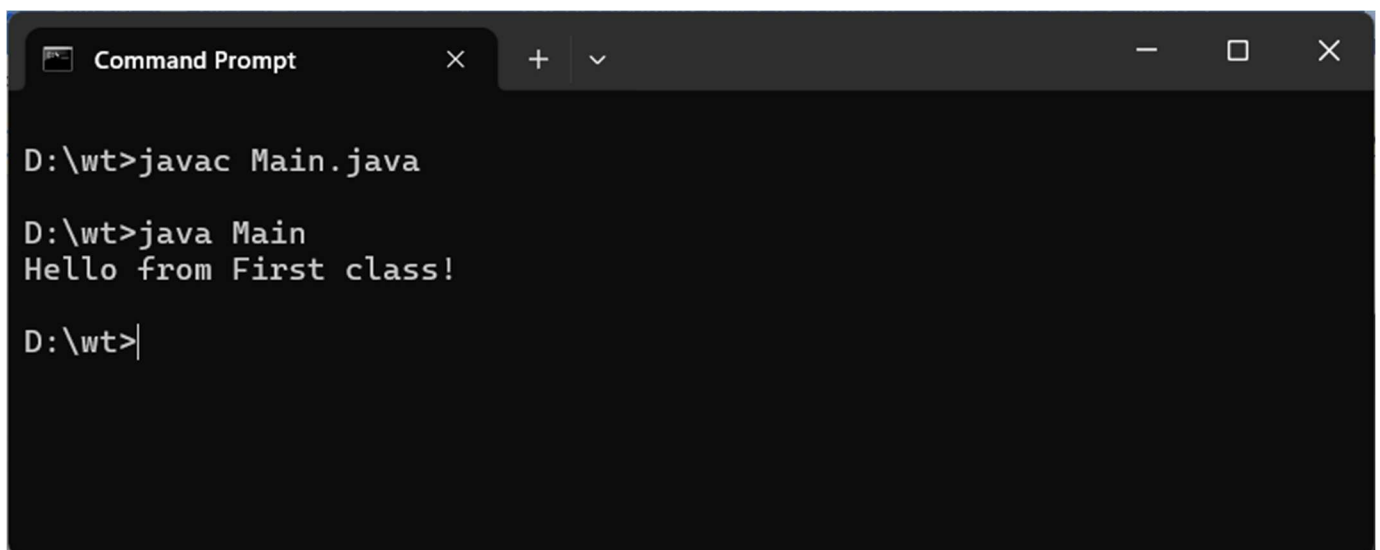
Program No. 3

Objective: Write a program in Java with two classes create a object of first class and call into another class (having main method).

Code Implementation (Source Code):

```
class First {  
    void display() {  
        System.out.println("Hello from First class!");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        First obj = new First();  
        obj.display();  
    }  
}
```

Output Screen:



```
Command Prompt  
D:\wt>javac Main.java  
D:\wt>java Main  
Hello from First class!  
D:\wt>|
```

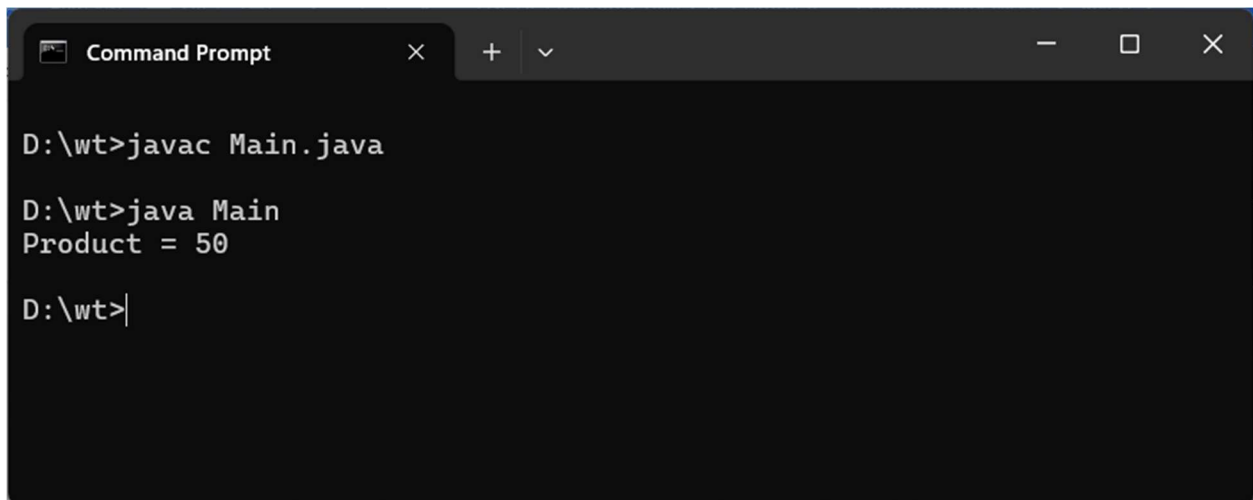
Program No. 4

Objective: Write a program in java to product of two numbers.

Code Implementation (Source Code):

```
public class Main {  
    public static void main(String[] args) {  
        int a = 5, b = 10;  
        int product = a * b;  
        System.out.println("Product = " + product);  
    }  
}
```

Output Screen:

A screenshot of a Windows Command Prompt window. The title bar shows 'Command Prompt' with standard window controls. The command prompt shows the following sequence of commands and output:
D:\wt>javac Main.java

D:\wt>java Main
Product = 50

D:\wt>|
The text is displayed in a monospaced font on a dark background.

Program No. 5

Objective: Write a program in java product of two numbers (Input by the user).

Code Implementation (Source Code):

```
import java.util.Scanner;

public class Main {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter first number: ");

        int a = sc.nextInt();

        System.out.print("Enter second number: ");

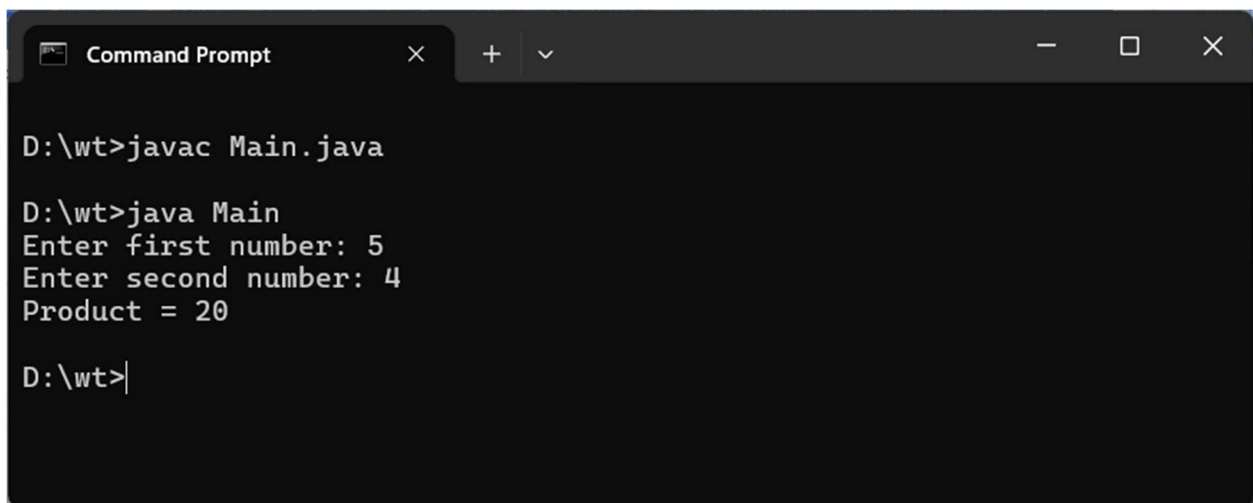
        int b = sc.nextInt();

        System.out.println("Product = " + (a * b));

    }

}
```

Output Screen:



```
D:\wt>javac Main.java

D:\wt>java Main
Enter first number: 5
Enter second number: 4
Product = 20

D:\wt>|
```

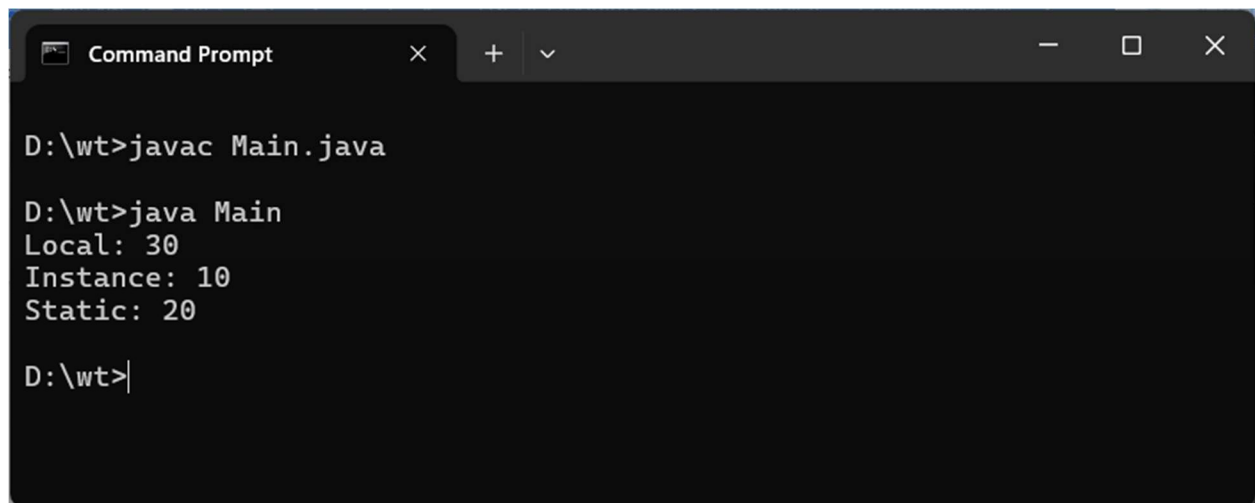

Program No. 6

Objective: Write a program in java to illustrate the concept of local, instance and static variable.

Code Implementation (Source Code):

```
public class Main {  
    int instanceVar = 10;  
    static int staticVar = 20;  
  
    void show() {  
        int localVar = 30;  
        System.out.println("Local: " + localVar);  
        System.out.println("Instance: " + instanceVar);  
        System.out.println("Static: " + staticVar);  
    }  
  
    public static void main(String[] args) {  
        Main obj = new Main();  
        obj.show();  
    }  
}
```

Output Screen:



A screenshot of a Windows Command Prompt window. The title bar at the top reads "Command Prompt" and includes standard window controls (minimize, maximize, close). The command prompt shows the following sequence of commands and output:

```
D:\wt>javac Main.java

D:\wt>java Main
Local: 30
Instance: 10
Static: 20

D:\wt>|
```

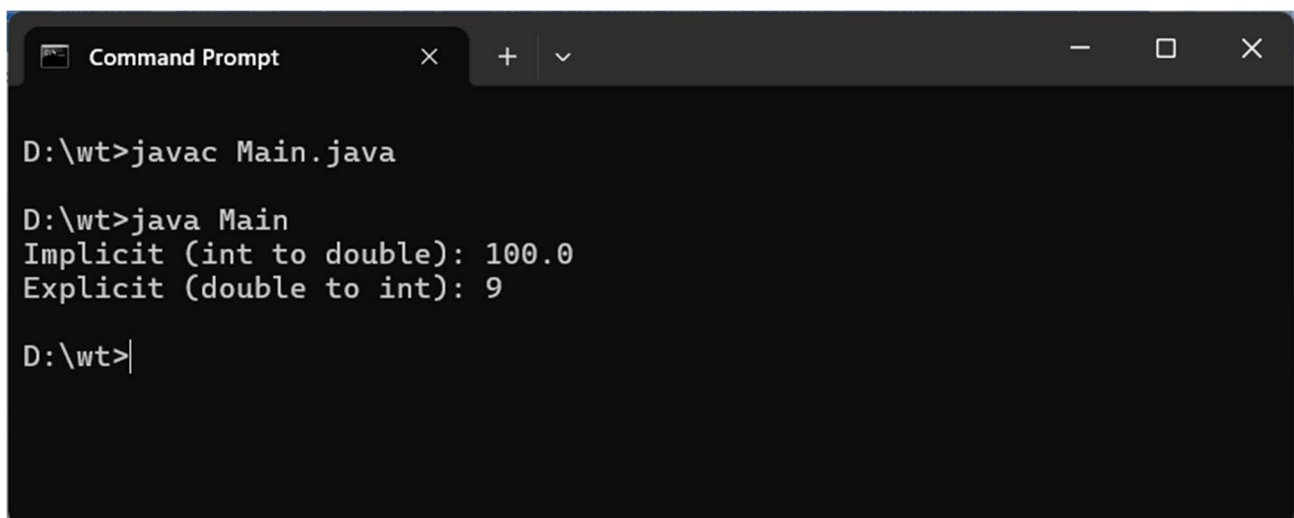
Program No. 7

Objective: Write a program in java to implement implicit and explicit type casting.

Code Implementation (Source Code):

```
public class Main {  
    public static void main(String[] args) {  
        int i = 100;  
        double d = i;  
        System.out.println("Implicit (int to double): " + d);  
  
        double x = 9.99;  
        int y = (int) x;  
        System.out.println("Explicit (double to int): " + y);  
    }  
}
```

Output Screen:



```
Command Prompt  
D:\wt>javac Main.java  
  
D:\wt>java Main  
Implicit (int to double): 100.0  
Explicit (double to int): 9  
  
D:\wt>|
```

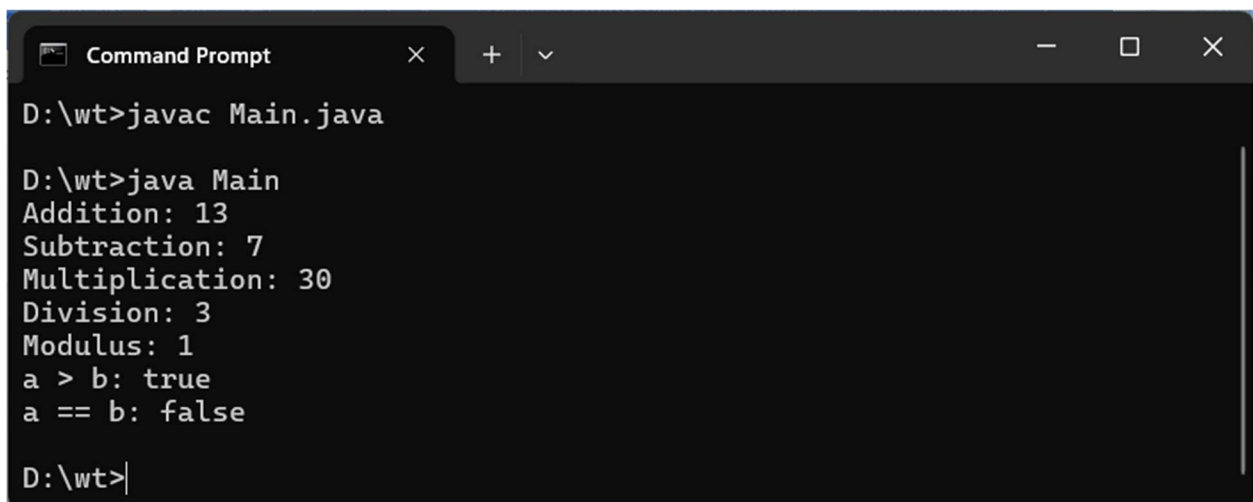
Program No. 8

Objective: Write a program in java to implement the various operators in java.

Code Implementation (Source Code):

```
public class Main {  
    public static void main(String[] args) {  
        int a = 10, b = 3;  
        System.out.println("Addition: " + (a + b));  
        System.out.println("Subtraction: " + (a - b));  
        System.out.println("Multiplication: " + (a * b));  
        System.out.println("Division: " + (a / b));  
        System.out.println("Modulus: " + (a % b));  
        System.out.println("a > b: " + (a > b));  
        System.out.println("a == b: " + (a == b));  
    }  
}
```

Output Screen:



```
Command Prompt  
D:\wt>javac Main.java  
D:\wt>java Main  
Addition: 13  
Subtraction: 7  
Multiplication: 30  
Division: 3  
Modulus: 1  
a > b: true  
a == b: false  
D:\wt>|
```

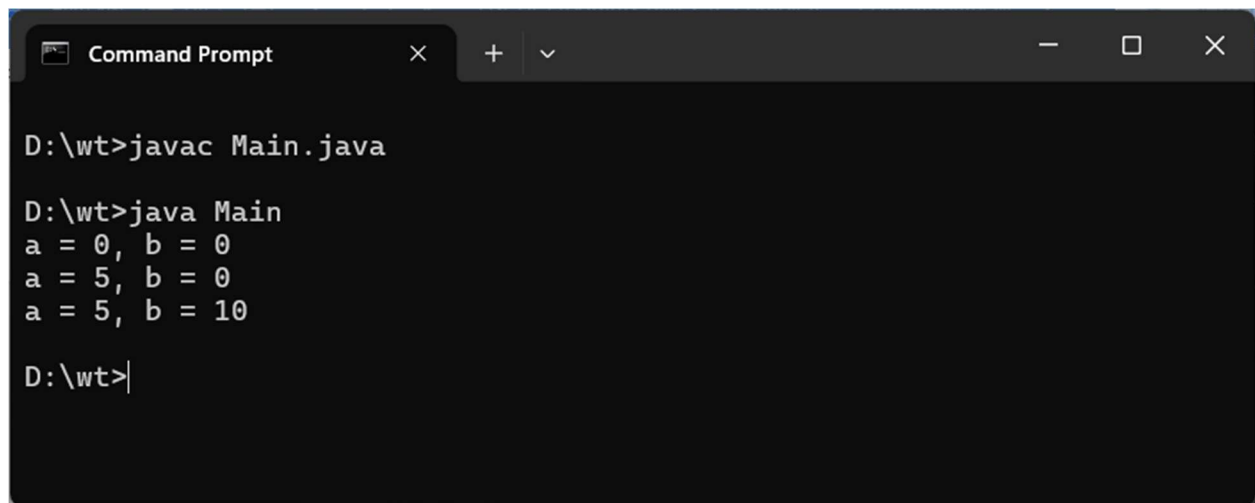
Program No. 9

Objective: Write a program in Java for constructor overloading.

Code Implementation (Source Code):

```
public class Main {  
    int a, b;  
    Main() {  
        a = 0; b = 0;  
    }  
    Main(int x) {  
        a = x;  
    }  
    Main(int x, int y) {  
        a = x; b = y;  
    }  
  
    void display() {  
        System.out.println("a = " + a + ", b = " + b);  
    }  
  
    public static void main(String[] args) {  
        new Main().display();  
        new Main(5).display();  
        new Main(5, 10).display();  
    }  
}
```

Output Screen:



```
Command Prompt
D:\wt>javac Main.java

D:\wt>java Main
a = 0, b = 0
a = 5, b = 0
a = 5, b = 10

D:\wt>
```

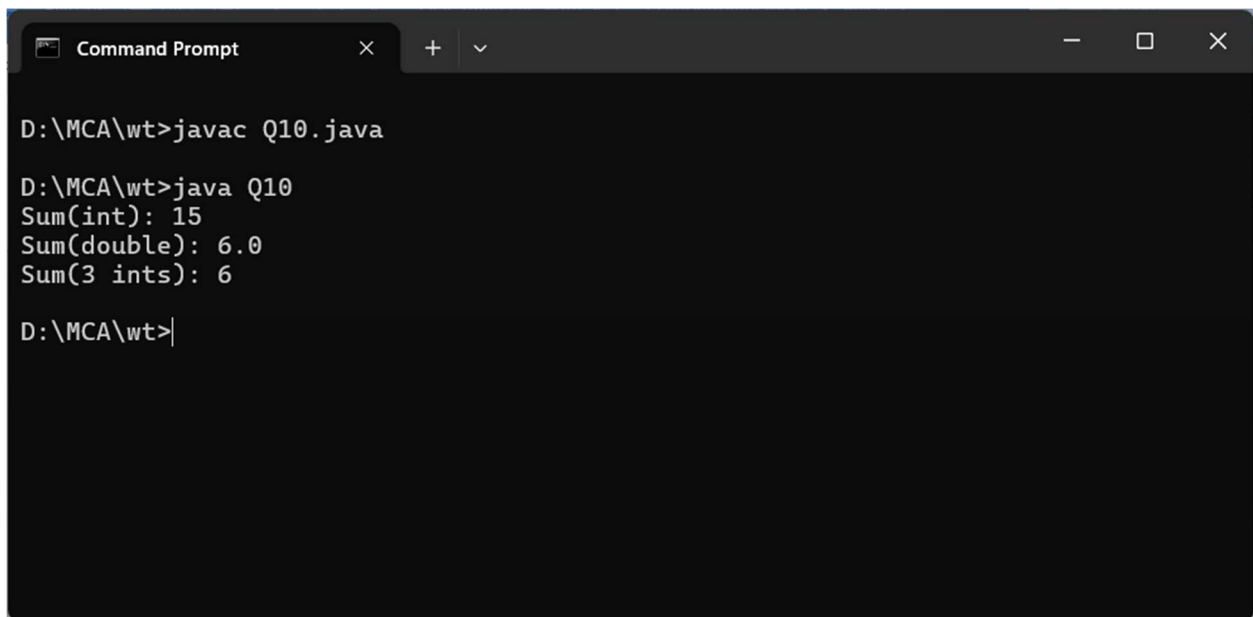
Program No. 10

Objective: Write a program in Java for method overloading.

Code Implementation (Source Code):

```
public class Q10 {  
    int add(int a, int b) {  
        return a + b;  
    }  
  
    double add(double a, double b) {  
        return a + b;  
    }  
  
    int add(int a, int b, int c) {  
        return a + b + c;  
    }  
  
    public static void main(String[] args) {  
        Q10 obj = new Q10();  
        System.out.println("Sum(int): " + obj.add(5, 10));  
        System.out.println("Sum(double): " + obj.add(2.5, 3.5));  
        System.out.println("Sum(3 ints): " + obj.add(1, 2, 3));  
    }  
}
```

Output Screen:



```
Command Prompt
D:\MCA\wt>javac Q10.java
D:\MCA\wt>java Q10
Sum(int): 15
Sum(double): 6.0
Sum(3 ints): 6
D:\MCA\wt>|
```

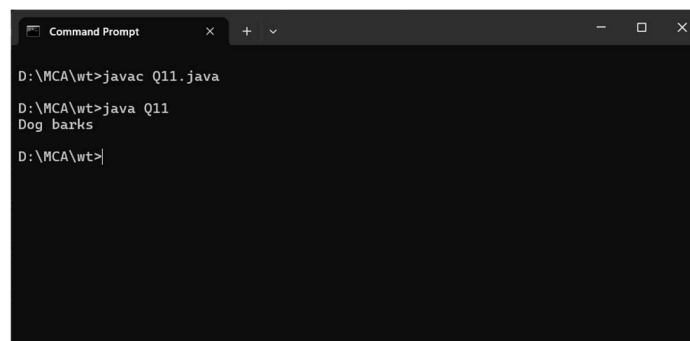

Program No. 11

Objective: Write a program in Java for method overriding.

Code Implementation (Source Code):

```
class Animal {  
    void sound() {  
        System.out.println("Animal makes a sound");  
    }  
}  
  
class Dog extends Animal {  
    @Override  
    void sound() {  
        System.out.println("Dog barks");  
    }  
}  
  
public class Q11 {  
    public static void main(String[] args) {  
        Animal a = new Dog();  
        a.sound();  
    }  
}
```

Output Screen:



```
Command Prompt  
D:\MCA\wt>javac Q11.java  
D:\MCA\wt>java Q11  
Dog barks  
D:\MCA\wt>
```

Program No. 12

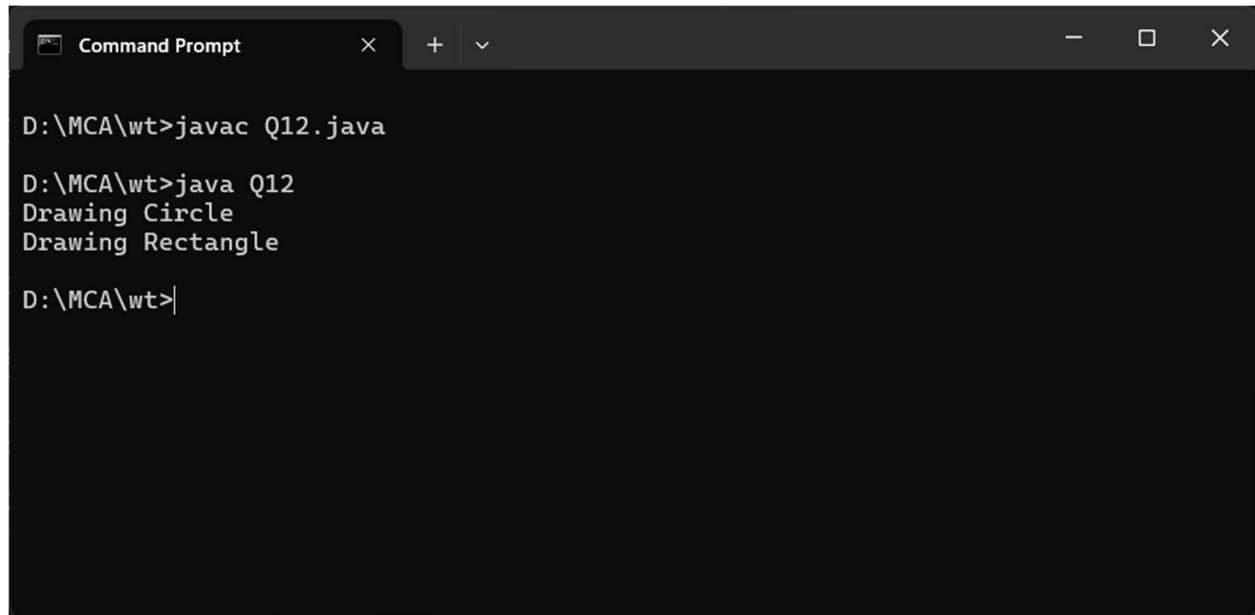
Objective: Write a program in java to show run time polymorphism (up casting).

Code Implementation (Source Code):

```
class Shape {  
    void draw() {  
        System.out.println("Drawing Shape");  
    }  
}  
  
class Circle extends Shape {  
    void draw() {  
        System.out.println("Drawing Circle");  
    }  
}  
  
class Rectangle extends Shape {  
    void draw() {  
        System.out.println("Drawing Rectangle");  
    }  
}  
  
public class Q12 {  
    public static void main(String[] args) {  
        Shape s;  
        s = new Circle();  
        s.draw();  
        s = new Rectangle();  
    }  
}
```

```
s.draw();  
}  
}
```

Output Screen:



The screenshot shows a Windows Command Prompt window with a dark background. The title bar at the top reads "Command Prompt" and includes standard window controls (minimize, maximize, close). The command prompt shows the following sequence of commands and output:

```
D:\MCA\wt>javac Q12.java  
D:\MCA\wt>java Q12  
Drawing Circle  
Drawing Rectangle  
D:\MCA\wt>|
```

Program No. 13

Objective: Write a program in java for access specifiers (all four).

Code Implementation (Source Code):

```
public class Q13 {  
    public int pubVar = 1;  
    protected int protVar = 2;  
    int defaultVar = 3;  
    private int privVar = 4;  
  
    void show() {  
        System.out.println("Public: " + pubVar);  
        System.out.println("Protected: " + protVar);  
        System.out.println("Default: " + defaultVar);  
        System.out.println("Private: " + privVar);  
    }  
  
    public static void main(String[] args) {  
        Q13 obj = new Q13();  
        obj.show();  
    }  
}
```

Output Screen:

```
Command Prompt
Microsoft Windows [Version 10.0.26200.8037]
(c) Microsoft Corporation. All rights reserved.

C:\Users\anshika>D:

D:\>cd wt

D:\wt>javac Q13.java
Q13.java:16: error: cannot find symbol
    obj.show();
      ^
  symbol:   method show()
  location: variable obj of type Main
1 error

D:\wt>javac Q13.java

D:\wt>java Q13
Public: 1
Protected: 2
Default: 3
Private: 4

D:\wt>
```

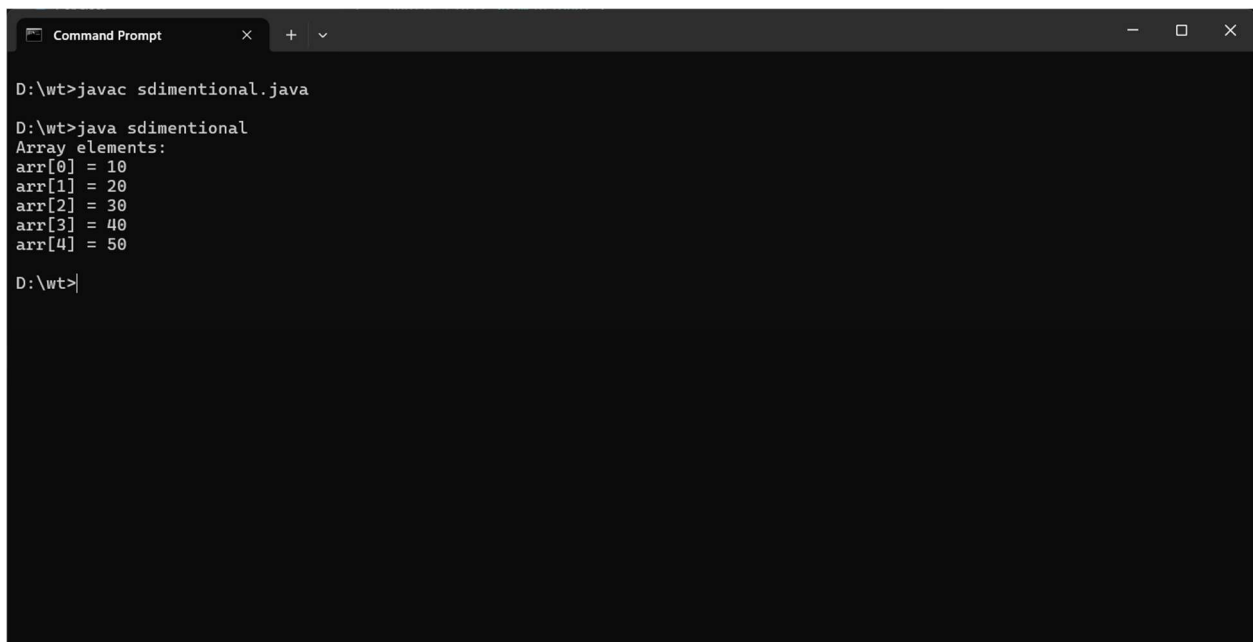
Program No. 14

Objective: Write a program in java to implement the single dimension array.

Code Implementation (Source Code):

```
public class sdimention {  
    public static void main(String[] args) {  
        int[] arr = {10, 20, 30, 40, 50};  
        System.out.println("Array elements:");  
        for (int i = 0; i < arr.length; i++) {  
            System.out.println("arr[" + i + "] = " + arr[i]);  
        }  
    }  
}
```

Output Screen:



```
Command Prompt  
D:\wt>javac sdimentional.java  
  
D:\wt>java sdimentional  
Array elements:  
arr[0] = 10  
arr[1] = 20  
arr[2] = 30  
arr[3] = 40  
arr[4] = 50  
  
D:\wt>
```

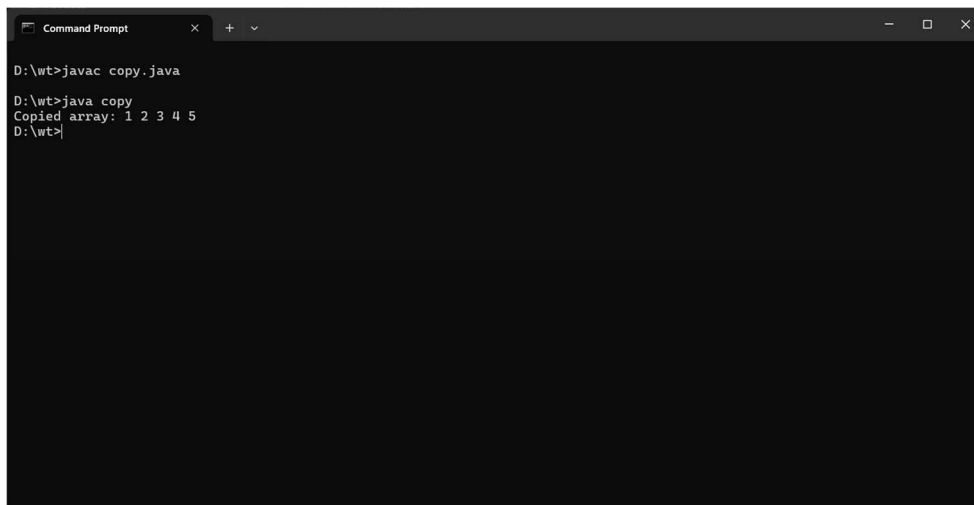
Program No. 15

Objective: Write a program in java to copy the elements from one array to another array.

Code Implementation (Source Code):

```
public class copy {  
    public static void main(String[] args) {  
        int[] src = {1, 2, 3, 4, 5};  
        int[] dest = new int[src.length];  
        for (int i = 0; i < src.length; i++) {  
            dest[i] = src[i];  
        }  
        System.out.print("Copied array: ");  
        for (int x : dest) {  
            System.out.print(x + " ");  
        }  
    }  
}
```

Output Screen:



```
Command Prompt
D:\wt>javac copy.java
D:\wt>java copy
Copied array: 1 2 3 4 5
D:\wt>
```

Program No. 16

Objective: Write a program in java to perform the addition and multiplication in 2-D array.

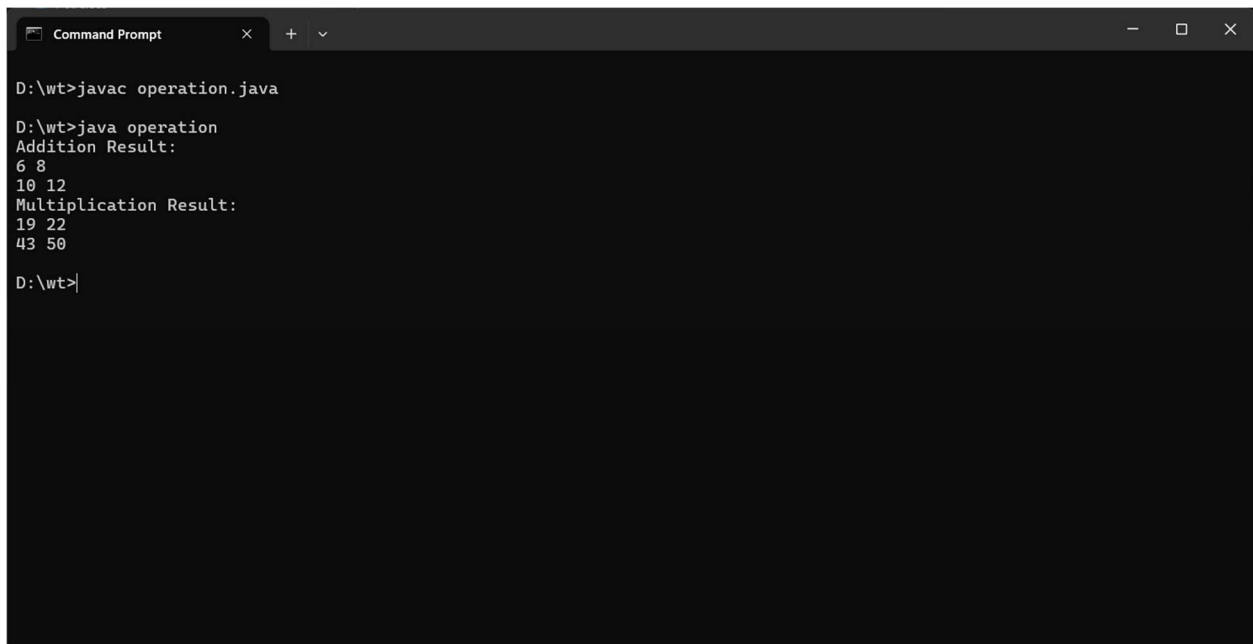
Code Implementation (Source Code):

```
public class operation {  
    public static void main(String[] args) {  
        int[][] a = {{1, 2}, {3, 4}};  
        int[][] b = {{5, 6}, {7, 8}};  
        int[][] sum = new int[2][2];  
        int[][] mul = new int[2][2];  
  
        for (int i = 0; i < 2; i++) {  
            for (int j = 0; j < 2; j++) {  
                sum[i][j] = a[i][j] + b[i][j];  
            }  
        }  
  
        for (int i = 0; i < 2; i++) {  
            for (int j = 0; j < 2; j++) {  
                for (int k = 0; k < 2; k++) {  
                    mul[i][j] += a[i][k] * b[k][j];  
                }  
            }  
        }  
  
        System.out.println("Addition Result:");  
        for (int[] row : sum) {
```



```
        for (int val : row) System.out.print(val + " ");  
        System.out.println();  
    }  
    System.out.println("Multiplication Result:");  
    for (int[] row : mul) {  
        for (int val : row) System.out.print(val + " ");  
        System.out.println();  
    }  
}  
}
```

Output Screen:



```
D:\wt>javac operation.java  
  
D:\wt>java operation  
Addition Result:  
6 8  
10 12  
Multiplication Result:  
19 22  
43 50  
  
D:\wt>
```

Program No. 17

Objective: Write a program in Java for simple inheritance.

Code Implementation (Source Code):

```
class Animal {  
    String name = "Animal";  
    void eat() {  
        System.out.println(name + " is eating.");  
    }  
}
```

```
class Dog extends Animal {  
    void bark() {  
        System.out.println("Dog is barking.");  
    }  
}
```

```
public class inheritance {  
    public static void main(String[] args) {  
        Dog d = new Dog();  
        d.name = "Dog";  
        d.eat();  
        d.bark();  
    }  
}
```

Output Screen:

```
Command Prompt
D:\wt>javac inheritance.java
D:\wt>java inheritance
Dog is eating.
Dog is barking.
D:\wt>
```

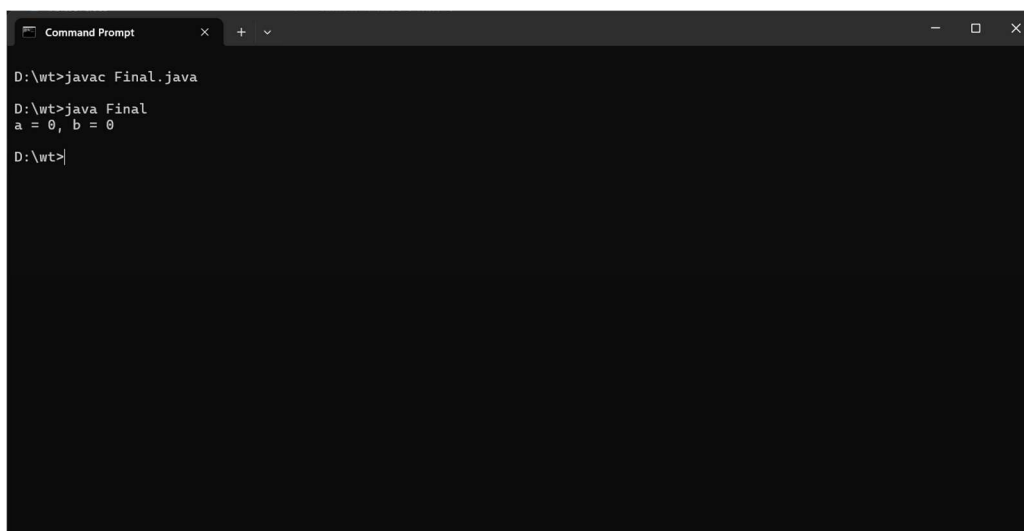
Program No. 18

Objective: Write a program in java for Final keyword.

Code Implementation (Source Code):

```
public class Final {  
    final int MAX = 100;  
  
    final void display() {  
        System.out.println("Final method called.");  
        System.out.println("Final variable MAX = " + MAX);  
    }  
  
    public static void main(String[] args) {  
        Main obj = new Main();  
        obj.display();  
    }  
}
```

Output Screen:



```
Command Prompt
D:\wt>javac Final.java
D:\wt>java Final
a = 0, b = 0
D:\wt>
```

Program No. 19

Objective: Write a program in java for super keyword.

Code Implementation (Source Code):

```
class Parent {  
    int x = 10;  
    void show() {  
        System.out.println("Parent show()");  
    }  
}  
  
class Child extends Parent {  
    int x = 20;  
    void show() {  
        super.show();  
        System.out.println("Parent x = " + super.x);  
        System.out.println("Child x = " + x);  
    }  
}  
  
public class superr {  
    public static void main(String[] args) {  
        Child c = new Child();  
        c.show();  
    }  
}
```

Output Screen:

```
Command Prompt
D:\wt>javac superr.java
D:\wt>java superr
Parent show()
Parent x = 10
Child x = 20
D:\wt>
```

Program No. 20

Objective: Write a program in Java chaining constructor.

Code Implementation (Source Code):

```
public class abc {  
    int a, b, c;  
    abc() {  
        this(1, 2);  
        System.out.println("No-arg constructor");  
    }  
    abc(int a, int b) {  
        this(a, b, 0);  
        System.out.println("Two-arg constructor");  
    }  
    abc(int a, int b, int c) {  
        this.a = a; this.b = b; this.c = c;  
        System.out.println("Three-arg: a=" + a + " b=" + b + " c=" + c);  
    }  
    public static void main(String[] args) {  
        new abc();  
    }  
}
```

Output Screen:

```
Command Prompt
D:\wt>javac abc.java
error: file not found: abc.java
Usage: javac <options> <source files>
use --help for a list of possible options

D:\wt>javac abc.java

D:\wt>java abc
Three-arg: a=1 b=2 c=0
Two-arg constructor
No-arg constructor

D:\wt>
```


Program No. 21

Objective: Write a program in Java for abstract method and abstract class.

Code Implementation (Source Code):

```
abstract class Shape {
    abstract double area();

    void display() {
        System.out.println("Area = " + area());
    }
}

class Circle extends Shape {
    double r;

    Circle(double r) { this.r = r; }

    double area() { return Math.PI * r * r; }
}

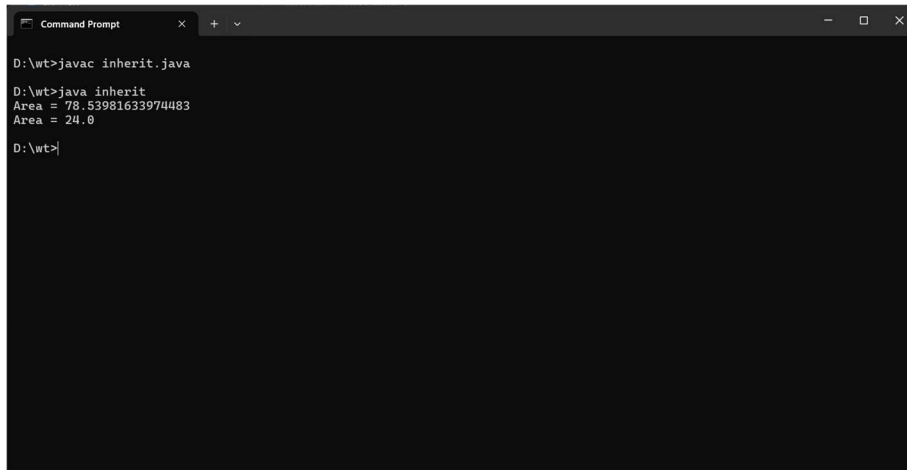
class Rectangle extends Shape {
    double l, w;

    Rectangle(double l, double w) { this.l = l; this.w = w; }

    double area() { return l * w; }
}

public class abstract {
    public static void main(String[] args) {
        Shape s1 = new Circle(5);
        Shape s2 = new Rectangle(4, 6);
        s1.display();
        s2.display();
    }
}
```

Output Screen:



```
Command Prompt
D:\wt>javac inherit.java
D:\wt>java inherit
Area = 78.53981633974483
Area = 24.0
D:\wt>
```

The image shows a Windows Command Prompt window with a dark background. The title bar at the top reads "Command Prompt" and includes standard window controls (minimize, maximize, close). The command prompt shows the following sequence of commands and output:

- Command: `D:\wt>javac inherit.java`
- Command: `D:\wt>java inherit`
- Output: `Area = 78.53981633974483`
- Output: `Area = 24.0`
- Command: `D:\wt>` (with a cursor at the end)

Program No. 22

Objective: Write a program in Java for interface.

Code Implementation (Source Code):

```
interface Drawable {
    void draw();
}

interface Resizable {
    void resize(int factor);
}

class Square implements Drawable, Resizable {
    int side = 5;
    public void draw() {
        System.out.println("Drawing Square with side = " + side);
    }
    public void resize(int factor) {
        side *= factor;
        System.out.println("Resized side = " + side);
    }
}

public class draw {
    public static void main(String[] args) {
        Square sq = new Square();
        sq.draw();
        sq.resize(3);
        sq.draw();
    }
}
```

Output Screen:

```
Command Prompt
D:\wt>javac drae.java
error: file not found: drae.java
Usage: javac <options> <source files>
use --help for a list of possible options

D:\wt>javac draw.java

D:\wt>java draw
Drawing Square with side = 5
Resized side = 15
Drawing Square with side = 15

D:\wt>|
```

Program No. 23

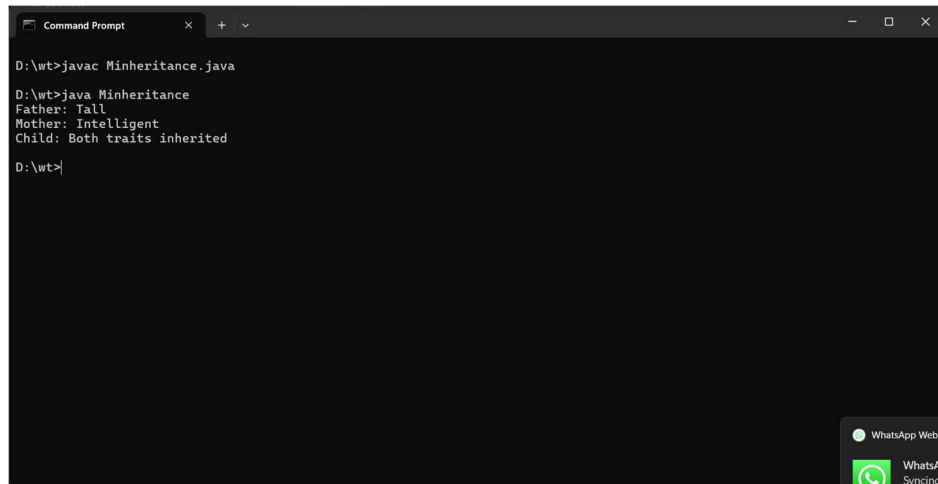
Objective: Write a program in Java multiple inheritance.

Code Implementation (Source Code):

```
interface Father {  
    default void tall() {  
        System.out.println("Father: Tall");  
    }  
}  
  
interface Mother {  
    default void intelligent() {  
        System.out.println("Mother: Intelligent");  
    }  
}  
  
class Child implements Father, Mother {  
    void myTraits() {  
        tall();  
        intelligent();  
        System.out.println("Child: Both traits inherited");  
    }  
}  
  
public class Minheritance {  
    public static void main(String[] args) {  
        Child c = new Child();  
        c.myTraits();  
    }  
}
```

```
}  
}
```

Output Screen:



```
D:\wt>javac Minheritance.java  
D:\wt>java Minheritance  
Father: Tall  
Mother: Intelligent  
Child: Both traits inherited  
D:\wt>
```

Program No. 24

Objective: Write a program in Java for Object Cloning (shallow and deep copy).

Code Implementation (Source Code):

```
class Address implements Cloneable {
    String city;
    Address(String city) { this.city = city; }
    protected Object clone() throws CloneNotSupportedException {
        return super.clone();
    }
}

class Student implements Cloneable {
    String name;
    Address addr;
    Student(String name, Address addr) {
        this.name = name; this.addr = addr;
    }

    protected Object clone() throws CloneNotSupportedException {
        return super.clone();
    }

    Student deepClone() throws CloneNotSupportedException {
        Student s = (Student) super.clone();
        s.addr = (Address) addr.clone();
        return s;
    }
}
```

```

public class cloning {

    public static void main(String[] args) throws CloneNotSupportedException {

        Address a = new Address("Delhi");

        Student s1 = new Student("Manu", a);

        Student s2 = (Student) s1.clone();

        Student s3 = s1.deepClone();

        s2.addr.city = "Mumbai";

        System.out.println("Shallow - s1 city: " + s1.addr.city);

        System.out.println("Shallow - s2 city: " + s2.addr.city);


        s3.addr.city = "Noida";

        System.out.println("Deep - s1 city: " + s1.addr.city);

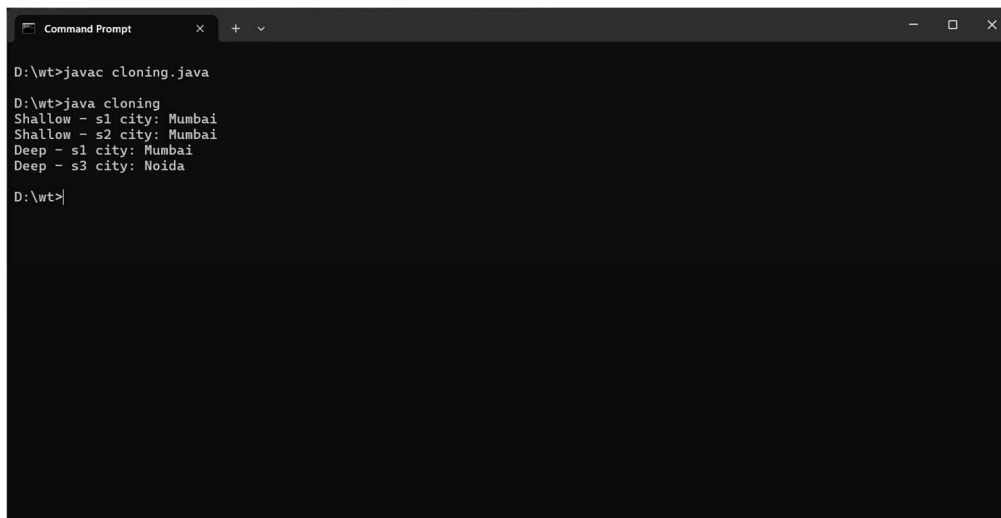
        System.out.println("Deep - s3 city: " + s3.addr.city);

    }

}

```

Output Screen:



```

Command Prompt
D:\wt>javac cloning.java
D:\wt>java cloning
Shallow - s1 city: Mumbai
Shallow - s2 city: Mumbai
Deep - s1 city: Mumbai
Deep - s3 city: Noida
D:\wt>

```


Program No. 25

Objective: Write a program in Java for Inner Classes (all types).

Code Implementation (Source Code):

```
public class program{
    class Inner {
        void show() { System.out.println("Regular Inner Class"); }
    }
    static class StaticNested {
        void show() { System.out.println("Static Nested Class"); }
    }

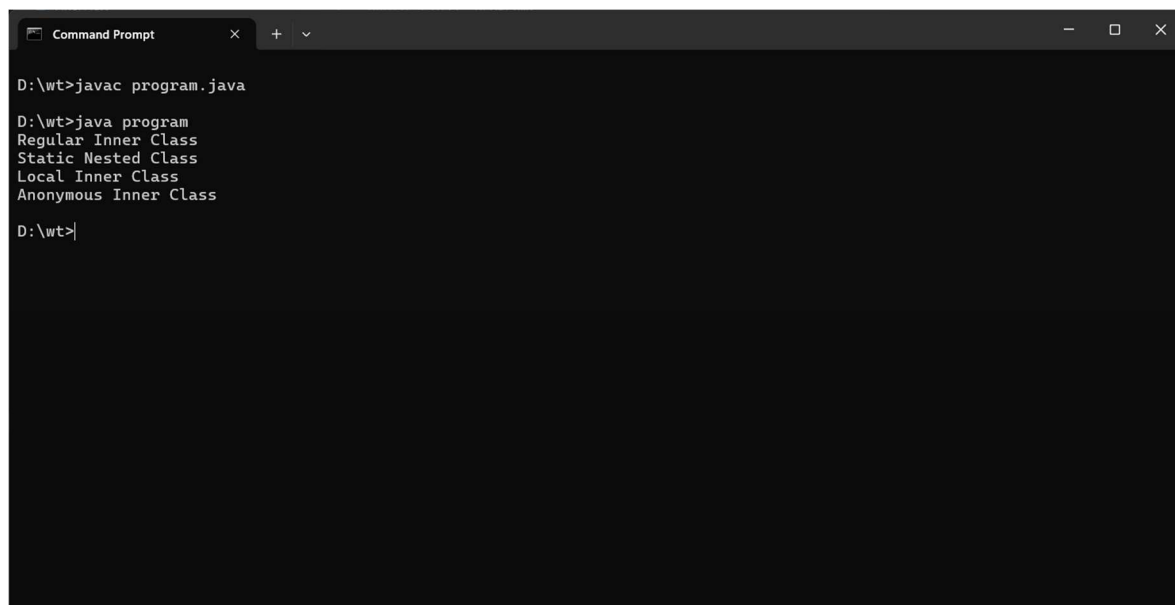
    void methodWithLocalClass() {
        class Local {
            void show() { System.out.println("Local Inner Class"); }
        }
        new Local().show();
    }

    public static void main(String[] args) {
        program outer = new program();
        outer.new Inner().show();
        new StaticNested().show();

        outer.methodWithLocalClass();
    }
}
```

```
Runnable r = new Runnable() {  
    public void run() { System.out.println("Anonymous Inner Class"); }  
};  
r.run();  
}  
}
```

Output Screen:



```
Command Prompt  
D:\wt>javac program.java  
D:\wt>java program  
Regular Inner Class  
Static Nested Class  
Local Inner Class  
Anonymous Inner Class  
D:\wt>
```

Program No. 26

Objective: Write a program in java to create the package (user defined package).

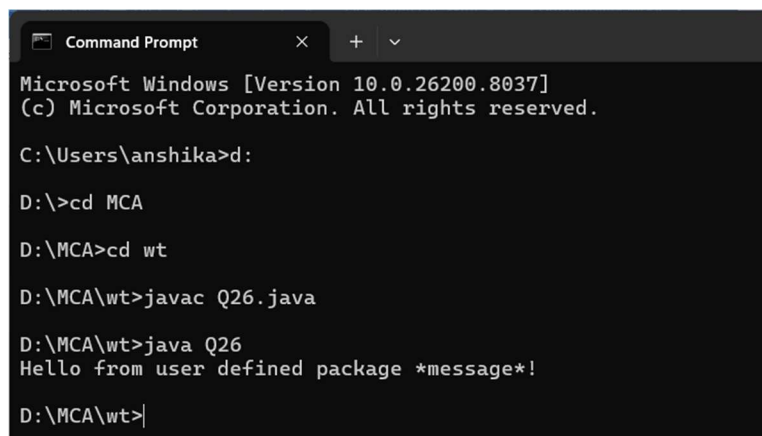
Code Implementation (Source Code):

```
package myPackage;  
  
public class Message {  
    public void show() {  
        System.out.println("Hello from user defined package *message*!");  
    }  
}
```

```
import myPackage.Message;
```

```
public class Q26 {  
    public static void main(String[] args) {  
        Message msg = new Message();  
        msg.show();  
    }  
}
```

Output Screen:



```
Microsoft Windows [Version 10.0.26200.8037]  
(c) Microsoft Corporation. All rights reserved.  
  
C:\Users\anshika>d:  
  
D:\>cd MCA  
  
D:\MCA>cd wt  
  
D:\MCA\wt>javac Q26.java  
  
D:\MCA\wt>java Q26  
Hello from user defined package *message*!  
  
D:\MCA\wt>|
```

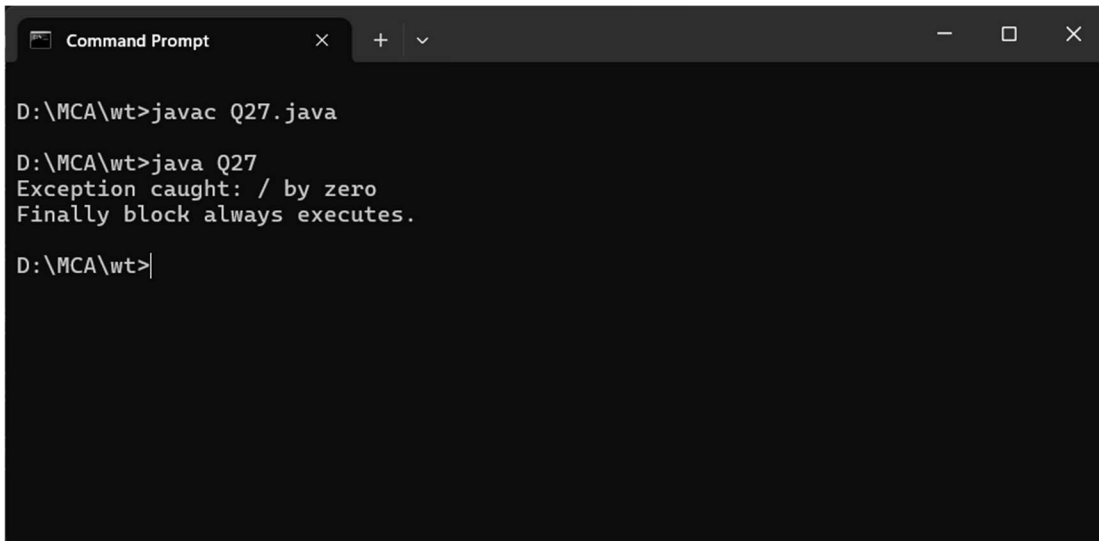
Program No. 27

Objective: Write a program in Java for exception handling by using try, catch and finally.

Code Implementation (Source Code):

```
public class Q27 {  
    public static void main(String[] args) {  
        try {  
            int a = 10, b = 0;  
            int result = a / b;  
            System.out.println("Result = " + result);  
        } catch (ArithmeticException e) {  
            System.out.println("Exception caught: " + e.getMessage());  
        } finally {  
            System.out.println("Finally block always executes.");  
        }  
    }  
}
```

Output Screen:



```
Command Prompt  
D:\MCA\wt>javac Q27.java  
D:\MCA\wt>java Q27  
Exception caught: / by zero  
Finally block always executes.  
D:\MCA\wt>|
```